

Analisi dei Bug nel Codice Sorgente MOLOCH

repository globo-bolam-moloch-at-lamma

Directory analizzata:

`sources/moloch/`

File analizzati:

<code>premoloch.F90</code>	pre-processore
<code>moloch.F90</code>	modello principale
<code>postmoloch.F90</code>	post-processore

Bug confermati: **9**

Critici	1
Alti	5
Medi	3

Indice

Nota metodologica	2
1 File: premoloch.F90	3
1.1 Bug 1 — <code>jp1</code> sempre uguale a 1	3
1.2 Bug 2 — Bound errato per <code>ip1</code>	3
1.3 Bug 3 — Precedenza operatori <code>.or.</code> / <code>.and.</code>	4
2 File: postmoloch.F90	5
2.1 Bug 4 — <code>ierr</code> non inizializzato in <code>rd_param_const_old</code>	5
2.2 Bug 5 — <code>orog_crossy</code> scritto al posto di <code>orog_crossx</code>	5
2.3 Bug 6 — Precedenza operatori nella condizione di <code>exit</code>	6
3 File: moloch.F90	8
3.1 Bug 7 — Tipo MPI errato in <code>disperse_int</code>	8
3.2 Bug 8 — <code>zprod</code> usato potenzialmente non inizializzato	8
3.3 Bug 9 — Divisione per zero nella convergenza iterativa	9
4 Tabella riepilogativa	11

Nota metodologica

Ogni file è stato letto integralmente e i bug identificati sono stati verificati direttamente sul codice sorgente prima di essere inclusi in questo report. Per ciascun bug vengono indicati: numero di riga esatto, codice problematico, spiegazione del problema, impatto a runtime e proposta di correzione.

I livelli di severità utilizzati sono:

- **CRITICO** — produce sempre risultati errati o crash garantito
- **ALTO** — produce dati corrotti o comportamento indefinito in condizioni plausibili
- **MEDIO** — logica scorretta in casi specifici, risultati silenziosamente sbagliati

File: premoloch.F90

Bug 1 — jp1 sempre uguale a 1

Severità: CRITICO

Riga: 1381

Codice problematico:

```

1379 elseif (input_model == 'COSMO') then
1380   do j = 1, nlat_inp
1381     jp1 = min(1, j+1)           ! BUG: sempre 1
1382   do i = 1, nlon_inp
1383     ip1 = min(nlon, i+1)
1384     u_inp_work(i,j) = 0.5*(u_inp(i,j,k) + u_inp(ip1,j,k))
1385     v_inp_work(i,j) = 0.5*(v_inp(i,j,k) + v_inp(i,jp1,k)) ! jp1 = sempre
                        1

```

Analisi:

La variabile jp1 viene calcolata come $\min(1, j+1)$. Poiché il ciclo esterno parte da $j = 1$, il valore $j+1$ è sempre ≥ 2 , e quindi $\min(1, j+1) = 1$ per ogni iterazione del loop.

La riga 1385 interpola la componente `v_inp` usando la media tra il punto (i,j,k) e il punto $(i,jp1,k)$: lo scopo è implementare lo staggering della griglia Arakawa-C per il vento meridionale. Con jp1 bloccato a 1, l'interpolazione non usa mai la riga adiacente $j+1$, ma sempre la prima riga della griglia, producendo campi di vento completamente errati per ogni $j > 1$.

Impatto a runtime: Il campo `v_inp_work` risulta scorretto per tutta la griglia eccetto la prima riga. Le condizioni al contorno verticali/laterali del modello MOLOCH derivanti dall'input COSMO saranno sistematicamente sbagliate.

Fix:

```

1 jp1 = min(nlat_inp, j+1)

```

Bug 2 — Bound errato per ip1

Severità: ALTO

Riga: 1383

Codice problematico:

```

1382 do i = 1, nlon_inp
1383   ip1 = min(nlon, i+1)           ! BUG: nlon e' la dimensione MOLOCH
1384   u_inp_work(i,j) = 0.5*(u_inp(i,j,k) + u_inp(ip1,j,k))

```

Analisi:

`nlon` è la dimensione longitudinale della griglia MOLOCH (output), mentre `u_inp` ha prima dimensione `nlon_inp` (griglia di input COSMO). Se $nlon > nlon_inp$ — situazione comune quando MOLOCH ha risoluzione più alta dell'input — allora `ip1` può superare `nlon_inp`, causando un accesso fuori dai limiti dell'array `u_inp`.

Impatto a runtime: Accesso fuori bounds su `u_inp`; in Fortran senza `-fcheck=bounds` il programma legge memoria arbitraria, producendo valori di vento zonale corrotti sull'ultima colonna.

Fix:

```
1 ip1 = min(nlon_inp, i+1)
```

Bug 3 — Precedenza operatori .or. / .and.**Severità: MEDIO****Righe: 299, 302****Codice problematico:**

```
297 if (input_format == 'grib2' .or. input_format == 'GRIB2') then
298   input_file(:) = "grib_000"
299 elseif (input_format == 'mhfb' .or. input_format == 'MHFB' &
300         .and. input_file_std == 2) then ! BUG: precedenza errata
301   input_file_atm(:) = "mhfb_000_atm"
302   input_file_soil(:) = "mhfb_000_soil"
303 elseif (input_format == 'mhfm' .or. input_format == 'MHFM' &
304         .and. input_file_std == 2) then ! BUG: stesso problema
```

Analisi:

In Fortran l'operatore `.and.` ha precedenza maggiore di `.or.`, pertanto la condizione alla riga 299 viene valutata come:

```
(input_format == 'mhfb') .or. (input_format == 'MHFB' .and. input_file_std == 2)
```

Ciò significa che quando `input_format == 'mhfb'` (minuscolo), il controllo `input_file_std == 2` viene completamente ignorato e il ramo viene eseguito indipendentemente dal valore di `input_file_std`. Il comportamento probabilmente atteso è che il controllo si applichi ad entrambe le varianti maiuscola e minuscola.

Impatto a runtime: Assegnazione dei nomi file sbagliata per certi valori di `input_file_std`; i file aperti successivamente potrebbero essere errati.

Fix:

```
1 elseif ((input_format == 'mhfb' .or. input_format == 'MHFB') &
2         .and. input_file_std == 2) then
```

File: postmoloch.F90

Bug 4 — ierr non inizializzato in rd_param_const_old

Severità: ALTO

Righe: 2849, 2875–2885

Codice problematico:

```

2847 integer :: nlon, nlat, nlev, nlevg, mhfr,          &
2848           nlon_local, nlat_local, nlev_local,      &
2849           iunit=11, ierr_open, ierr, ...           ! ierr dichiarato senza
           valore
2850
2851 ! ... lettura file ...
2852
2853 if (nlon_local /= nlon) ierr=ierr+1 ! BUG: ierr e' indeterminato
2854 if (nlat_local /= nlat) ierr=ierr+1
2855 if (nlevg_local /= nlevg) ierr=ierr+1
2856 if (dlon_local /= dlon) ierr=ierr+1
2857 if (dlat_local /= dlat) ierr=ierr+1
2858 if (x0_local /= x0) ierr=ierr+1
2859 if (y0_local /= y0) ierr=ierr+1
2860 if (alon0_local /= alon0) ierr=ierr+1
2861 if (alat0_local /= alat0) ierr=ierr+1
2862
2863 if (ierr /= 0) then ! confronto con valore garbage
2864   print *, "Error in header parameters ..."
2865   stop
2866 endif

```

Analisi:

La variabile locale `ierr` viene dichiarata senza valore iniziale. In Fortran le variabili locali automatiche non sono inizializzate a zero dal linguaggio: il loro valore alla prima esecuzione è indeterminato (dipende da cosa si trovava nello stack in quel momento).

Le istruzioni `ierr=ierr+1` incrementano un valore garbage, quindi il test finale `if (ierr /= 0)` può scattare anche quando tutti i parametri del file sono corretti, causando uno `stop` ingiustificato.

A confronto, la routine `rd_param_const` (la versione più recente dello stesso file) inizializza correttamente `ierr=0` prima degli incrementi.

Impatto a runtime: Il programma può terminare con errore anche con input perfettamente valido, oppure — se il valore garbage è negativo — non rilevare effettive discordanze nei parametri.

Fix: aggiungere una riga di inizializzazione dopo la `read`:

```

1 read (iunit) nlon_local, nlat_local, ...
2 ierr = 0 ! <-- aggiungere questa riga
3 if (nlon_local /= nlon) ierr=ierr+1

```

Bug 5 — `orog_crossy` scritto al posto di `orog_crossx`

Severità: ALTO**Riga: 2538****Codice problematico:**

```

2520 ! Sezioni N-S (ncrossy): loop corretto
2521 do j=1,ncrossy
2522   ...
2523   do jlat=1,nlat
2524     orog_crossy(jlat,j) = zorogr(loncr(j),jlat)    ! OK
2525   enddo
2526 enddo
2527
2528 ! Sezioni E-W (ncrossx): BUG - scrive su orog_crossy invece di
      orog_crossx
2529 do j=1,ncrossx
2530   ...
2531   do jlon=1,nlon
2532     orog_crossy(jlon,j) = zorogr(jlon,latcr(j))    ! BUG: doveva essere
      orog_crossx
2533   enddo
2534 enddo

```

Analisi:

Errore di copia-incolla: il secondo blocco, che elabora le sezioni E-W (`ncrossx`), scrive sull'array `orog_crossy` invece di `orog_crossx`. Di conseguenza:

- `orog_crossx` non viene mai popolato (valori indeterminati o zero)
- `orog_crossy` viene sovrascritto con dati dimensionalmente incompatibili: il primo indice dovrebbe essere `nlat` per `orog_crossy`, ma viene usato `jlon` che va fino a `nlon`, potenzialmente diverso

Impatto a runtime: Le sezioni verticali E-W delle cross-section mostrano orografia nulla o corrotta; possibile accesso fuori bounds se `nlon > nlat`.

Fix:

```

1 orog_crossx(jlon,j) = zorogr(jlon,latcr(j))

```

Bug 6 — Precedenza operatori nella condizione di exit**Severità: MEDIO****Riga: 341–342****Codice problematico:**

```

340 do icross = 1,10
341   if (lonini_cross(icross) == valmiss .or.                &
342       latini_cross(icross) == valmiss .or.                &
343       lonfin_cross(icross) == valmiss .and.               &
344       latfin_cross(icross) == valmiss) exit    ! BUG: precedenza .and.

```

Analisi:

Per la stessa ragione del Bug 3, l'espressione viene valutata come:

```
(lonini==valmiss) .or. (latini==valmiss) .or.  
(lonfin==valmiss .and. latfin==valmiss)
```

Il ciclo non esce quando `lonfin_cross` è mancante ma `latfin_cross` non lo è (o viceversa), lasciando elaborare cross-section definite solo parzialmente.

Fix: sostituire l'ultimo `.and.` con `.or.:`

```
1 if (lonini_cross(icross) == valmiss .or. &  
2   latini_cross(icross) == valmiss .or. &  
3   lonfin_cross(icross) == valmiss .or. &  
4   latfin_cross(icross) == valmiss) exit
```


File: moloch.F90

Bug 7 — Tipo MPI errato in disperse_int

Severità: ALTO

Righe: 3387, 3390

Codice problematico:

```

3380  ! La subroutine si chiama disperse_int e gestisce dati INTEGER
3381  if (myid == 0) then
3382    ilfield(1:nlon,1:nlat) = igfield(1:nlon,1:nlat)  ! igfield: INTEGER
3383    do jpr = 1, nprocsx*nprocsy-1
3384      ...
3385      call mpi_send (igfield(sx:ex,sy:ey), count, mpi_real, &  ! BUG:
                 mpi_real
3386                      jpr, tag, comm, error)
3387    enddo
3388  else
3389    call mpi_recv (ilfield, count, mpi_real, &  ! BUG: mpi_real
3390                  0, tag, comm, status, error)
3391  endif

```

Analisi:

La subroutine `disperse_int` è esplicitamente progettata per distribuire campi interi (`igfield` e `ilfield` sono array di tipo `integer`). Tuttavia, sia la `mpi_send` che la `mpi_recv` specificano `mpi_real` come tipo di dato MPI.

In MPI il tipo specifica la dimensione e l'interpretazione dei byte da trasmettere. Se `sizeof(integer) ≠ sizeof(real)` (possibile su alcune architetture o con opzioni di compilazione specifiche) il numero di byte trasferiti non corrisponde, e i dati ricevuti risultano corrotti. Anche quando le dimensioni coincidono, l'uso del tipo sbagliato viola la specifica MPI ed è potenzialmente non portabile.

Impatto a runtime: Corruzione silente dei campi interi distribuiti via MPI in esecuzione parallela.

Fix:

```

1  call mpi_send (igfield(sx:ex,sy:ey), count, mpi_integer, jpr, tag, comm,
    error)
2  ...
3  call mpi_recv (ilfield, count, mpi_integer, 0, tag, comm, status, error)

```

Bug 8 — zprod usato potenzialmente non inizializzato

Severità: ALTO

Righe: 5896–5905

Codice problematico:

```

5896  pden = 1.
5897  do k = 1, kmi
5898    pden = pden*(1.-zcol(kmin(k)))
5899  enddo

```

```

5900
5901 ! zprod assegnato SOLO se pden != 0
5902 if (pden.ne.0.) zprod = pnum/pden
5903
5904 ! zprod usato SEMPRE, anche se non e' stato assegnato
5905 cloudt(jlon,jlat) = max (0., 1.-zprod)
5906 cloudt(jlon,jlat) = min (cloudt(jlon,jlat), 1.)

```

Analisi:

pden parte da 1.0 e viene moltiplicato per (1.-zcol(kmin(k))) ad ogni iterazione. Se almeno un elemento zcol(kmin(k)) vale esattamente 1.0, pden diventa 0 e l'assegnazione di zprod viene saltata. La riga 5905 usa zprod senza verificare se sia stato effettivamente calcolato: in quel caso conterrà il valore dell'iterazione precedente del loop su (jlon, jlat), producendo un valore di cloudt copiato da un punto di griglia adiacente. Alla prima iterazione del loop (o se la variabile non è inizializzata dal compilatore) il comportamento è completamente indefinito.

Impatto a runtime: Copertura nuvolosa totale (cloudt) erroneamente copiata da celle di griglia adiacenti nei punti dove la colonna d'aria è completamente satura.

Fix: inizializzare zprod prima del controllo:

```

1 zprod = 0.
2 if (pden.ne.0.) zprod = pnum/pden
3 cloudt(jlon,jlat) = max (0., 1.-zprod)

```

Bug 9 — Divisione per zero nella convergenza iterativa

Severità: MEDIO

Righe: 5027–5028

Codice problematico:

```

5020 if(zri.gt.0.) then           ! funzioni di stabilita' Holtslag
5021   zal = .2 + 4.*zri
5022   zpsim = psism(zal, zal*zrgm/zza)
5023   zpsih = psish(zal, zal*zrgt/zza)
5024   do jiter = 1, 10
5025     zalm = zal
5026     zal = zri*(zalam-zpsim)**2/(zalat-zpsih)   ! zal puo' essere 0
5027     error = abs(zal-zalm)/zal                 ! BUG: divisione per
                                                zero
5028     if (error.lt.1.e-2) go to 1
5029     ...
5030   enddo

```

Analisi:

Nel ciclo iterativo per la convergenza delle funzioni di stabilità atmosferica di Holtslag, la variabile zal viene ricalcolata alla riga 5027. Se (zalam-zpsim) == 0, allora zal = 0 e la riga successiva esegue una divisione per zero.

La condizione zri.gt.0. garantisce che il numero di Richardson sia positivo, ma non esclude che il numeratore (zalam-zpsim)**2 si annulli durante le iterazioni, specialmente nelle prime iterazioni quando i valori di zpsim potrebbero ancora non essere convergiti.

Impatto a runtime: Floating-point exception (FPE) con `-ffpe-trap=zero` o valore Inf/NaN silente che si propaga nel calcolo dello strato limite.

Fix:

```
1 if (zal /= 0.) then
2   error = abs(zal-zalm)/zal
3 else
4   error = abs(zal-zalm)
5 endif
```

Tabella riepilogativa

#	File	Riga	Severità	Problema
1	premoloch.F90	1381	CRITICO	<code>min(1, j+1) ⇒ jp1</code> sempre 1; staggering <code>v_inp</code> sempre sbagliata
2	premoloch.F90	1383	ALTO	bound <code>nlon</code> invece di <code>nlon_inp</code> ; accesso fuori bounds su <code>u_inp</code>
3	premoloch.F90	299, 302	MEDIO	precedenza <code>.or./and.</code> ; check <code>input_file_std</code> bypassato
4	postmoloch.F90	2875–2885	ALTO	<code>ierr</code> non inizializzato in <code>rd_param_const_old</code>
5	postmoloch.F90	2538	ALTO	<code>orog_crossy</code> scritto invece di <code>orog_crossx</code>
6	postmoloch.F90	341–342	MEDIO	precedenza <code>.or./and.</code> ; condizione di exit cross-section errata
7	moloch.F90	3387, 3390	ALTO	<code>mpi_real</code> per dati interi in <code>disperse_int</code>
8	moloch.F90	5903–5905	ALTO	<code>zprod</code> usato senza inizializzazione quando <code>pden == 0</code>
9	moloch.F90	5027–5028	MEDIO	divisione per zero se <code>zal == 0</code> nel loop di convergenza

Totale: 1 critico • 5 alti • 3 medi